Exercise 4: dbt on Snowflake

Objective

Connect the dbt project from S10 (originally running on DuckDB) to Snowflake. Verify that the same SQL models work without changes — only the `profiles.yml` connection file is different.

Prerequisites

- Exercise 3 completed (or at least Exercise 1 + Exercise 2)
- S10 dbt project working locally (e.g., at `~/dbt-class/airbnb/`)
- Python virtual environment from S10
- **Account identifier** from Exercise 0 (e.g., `xy12345.eu-west-1`)
- **Password** from Exercise 1 (`changeme` unless you changed it)

Part A — Snowflake objects recap

The objects you created in Exercise 1 are what dbt will use. Quick check in Snowsight:

```
SHOW WAREHOUSES LIKE 'WH_DBT_DEV';
SHOW USERS LIKE 'DBT_USER';
SHOW ROLES LIKE 'TRANSFORM_ROLE';
SHOW DATABASES LIKE 'AIRBNB';
```

Each should return one row. If any is missing, go back to Exercise 1.

Also verify that `dbt_user` can log in. In Snowsight (logged in as `ACCOUNTADMIN`) run:

```
-- Allow dbt_user to create objects in the schema (already granted in Ex1,
-- but run again to be safe in case of ALTER TABLE issues)
GRANT ALL ON SCHEMA AIRBNB.DEV TO ROLE TRANSFORM_ROLE;
GRANT ALL ON ALL TABLES IN SCHEMA AIRBNB.DEV TO ROLE TRANSFORM_ROLE;
GRANT ALL ON ALL VIEWS IN SCHEMA AIRBNB.DEV TO ROLE TRANSFORM_ROLE;
```

Part B — Connect dbt to Snowflake

Step 1: Install the Snowflake adapter

Open your terminal and activate the S10 virtual environment:

```
cd ~/dbt-class
source .venv/bin/activate
```

> *Path may differ if you cloned the project elsewhere. Replace `~/dbt-class` with the actual path to your S10 project.*

Install the adapter (it will coexist with `dbt-duckdb`):

```
pip install dbt-snowflake
```

Verify:

```
dbt --version
```

Expected output includes both adapters:

```
Plugins:
  - duckdb:    1.9.x
  - snowflake: 1.9.x
```

Step 2: Locate profiles.yml

The `profiles.yml` file lives **outside** your dbt project.

- **macOS / Linux**: `~/.dbt/profiles.yml`
- **Windows**: `%USERPROFILE%\.dbt\profiles.yml`
- **WSL**: `/home/<username>/.dbt/profiles.yml` (the Linux `~`, not the Windows host)

If the directory does not exist:

```
mkdir -p ~/.dbt
touch ~/.dbt/profiles.yml
```

Step 3: Edit profiles.yml

Open `~/.dbt/profiles.yml` in your editor. Replace (or add) the `airbnb` entry:

```yaml
airbnb:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: "<YOUR_ACCOUNT_IDENTIFIER>"
      user: dbt_user
      password: "{{ env_var('SF_PASSWORD') }}"
      role: TRANSFORM_ROLE
      database: AIRBNB
      warehouse: WH_DBT_DEV
      schema: DEV
      threads: 4
```

Replace `<YOUR_ACCOUNT_IDENTIFIER>` with the value from Exercise 0 (e.g., `xy12345.eu-west-1`).

Step 4: Export the password as an environment variable

`env_var('SF_PASSWORD')` reads from the shell environment. Export it:

```
# macOS / Linux
export SF_PASSWORD='changeme'

# Windows PowerShell
$env:SF_PASSWORD = 'changeme'
```

> ***Do NOT commit the password*** *to Git. That is why we use* `env_var()` *instead of hardcoding it in* `profiles.yml` *.*
>
> ***In real production****: use* ***key-pair authentication*** *(RSA) instead of a password.* `dbt-snowflake` *supports it via* `private_key_path:` *or* `private_key:` *. Out of scope for this class.*

Part C — Run and verify

Step 5: Test the connection

```
cd ~/dbt-class/airbnb
dbt debug
```

Expected output (abbreviated):

```
profiles.yml file [OK found and valid]
dbt_project.yml file [OK found and valid]
Connection:
  account:   xy12345.eu-west-1
  user:      dbt_user
  database:  AIRBNB
  warehouse: WH_DBT_DEV
  role:      TRANSFORM_ROLE
  schema:    DEV
Connection test: [OK connection ok]
All checks passed!
```

If it fails, see **Troubleshooting** at the end.

Step 6: Load seed data

```
dbt seed
```

Expected: 3 CSV seeds (`raw_listings` , `raw_hosts` , `raw_reviews`) loaded as tables into `AIRBNB.DEV` .

> *If your S10 project did not use* `dbt seed` *, the raw tables you created in Exercise 2 are already in* `AIRBNB.DEV` *. Skip this step and continue.*

Step 7: Run the models

```
dbt run
```

Expected output (abbreviated):

```
1 of 5 OK created sql view model  AIRBNB.DEV.src_listings       [SUCCESS in 1.2s]
2 of 5 OK created sql view model  AIRBNB.DEV.src_hosts          [SUCCESS in 1.1s]
3 of 5 OK created sql view model  AIRBNB.DEV.src_reviews        [SUCCESS in 1.0s]
4 of 5 OK created sql table model AIRBNB.DEV.fct_listings       [SUCCESS in 2.5s]
5 of 5 OK created sql incremental AIRBNB.DEV.fct_reviews_daily  [SUCCESS in 3.2s]

Done. PASS=5 WARN=0 ERROR=0 SKIP=0 TOTAL=5
```

The output mirrors S10 but the models live in Snowflake now.

Step 8: Verify in Snowflake

Back in Snowsight:

```sql
USE WAREHOUSE WH_DBT_DEV;
USE DATABASE AIRBNB;
USE SCHEMA DEV;

SHOW OBJECTS IN SCHEMA AIRBNB.DEV;
```

You should see the raw tables plus the views and tables dbt created: `src_listings`, `src_hosts`, `src_reviews`, `fct_listings`, `fct_reviews_daily`.

Query the mart:

```sql
SELECT listing_name, neighbourhood, price_usd, host_name, is_superhost
FROM fct_listings
ORDER BY price_usd DESC;
```

Same 8 rows as in S10, with listing + host joined.

Step 9: Check what dbt did under the hood

Go to **Monitoring > Query History** in Snowsight. Filter by the last 5 minutes and `dbt_user`.

You will see every SQL statement dbt executed:

- `CREATE OR REPLACE VIEW AIRBNB.DEV.src_listings AS ...`
- `CREATE OR REPLACE TABLE AIRBNB.DEV.fct_listings AS ...`

This is one of Snowflake's advantages over DuckDB: full visibility into what dbt is doing, including execution time and bytes scanned per query.

Step 10 (optional): Tests and docs

If your S10 project has tests:

```
dbt test
```

Documentation:

```
dbt docs generate
dbt docs serve
```

Opens at http://localhost:8080. Same DAG as S10, now pointing at Snowflake.

What changed from S10 to S11?

```
                        S10 (DuckDB)         S11 (Snowflake)
                        ____________         _______________

profiles.yml            type: duckdb         type: snowflake
                        path: airbnb.duckdb  account: xy12345...
                                             user: dbt_user
                                             role: TRANSFORM_ROLE
                                             warehouse: WH_DBT_DEV

models/*.sql            (identical)          (identical)
seeds/*.csv             (identical)          (identical)
snapshots/*.sql         (identical)          (identical)
schema.yml              (identical)          (identical)

dbt run output          VIEW/TABLE in DuckDB  VIEW/TABLE in Snowflake
```

The only file that changed is `profiles.yml`.

Troubleshooting

`dbt debug` fails: "Incorrect username or password"

- Verify `SF_PASSWORD` is exported in the same shell where you run dbt: `echo $SF_PASSWORD` (macOS/Linux)
- Confirm the password of `dbt_user` is what you think: re-run `ALTER USER dbt_user SET PASSWORD = 'changeme';` in Snowsight.

`dbt debug` fails: "Could not connect to Snowflake"

Most common cause: the **account identifier** is wrong. Formats:

- New regional URL: `xy12345.eu-west-1` (from Snowsight URL)
- Organization-account: `ORG_NAME-ACCOUNT_NAME` (no region)
- Legacy locator: `xy12345` (without region, for old accounts)

Try running in Snowsight:

```
SELECT CURRENT_ACCOUNT() || '.' || REPLACE(CURRENT_REGION(), 'AWS_', '');
```

Use that value in `profiles.yml`.

`dbt run` fails: "Insufficient privileges"

The `TRANSFORM_ROLE` is missing a grant. Re-run the full set from **Part A** in Snowsight.

`dbt run` fails: "Warehouse WH_DBT_DEV is suspended"

Normally the warehouse auto-resumes, but if not: `ALTER WAREHOUSE WH_DBT_DEV RESUME;` in Snowsight.

Checklist

- [ ] `dbt-snowflake` installed and visible in `dbt --version`
- [ ] `profiles.yml` edited with your account identifier
- [ ] `SF_PASSWORD` exported in the shell
- [ ] `dbt debug` passes
- [ ] `dbt run` creates the 5 models in Snowflake
- [ ] Snowsight shows the new tables/views under `AIRBNB.DEV`

Key Concepts

Concept	What it means
dbt adapter	Plugin that connects dbt to a specific warehouse (dbt-duckdb, dbt-snowflake)
profiles.yml	Connection configuration. Lives outside the dbt project (~/.dbt/)
Account identifier	Unique Snowflake address, usually `<account>.<region>`
env_var()	dbt Jinja function to read from shell environment
Key-pair auth	Production-grade alternative to password auth (not used here)

Questions

1. Why is `SF_PASSWORD` loaded via `env_var()` instead of written literally in `profiles.yml`?
2. In production, would you use DuckDB or Snowflake? Why?
3. How would you deploy this project to a `PROD` schema instead of `DEV`? (Hint: look at the `target` concept in profiles.yml.)
4. Your CI/CD pipeline needs to run `dbt build` against Snowflake on every pull request. How would you provide the password without hardcoding it?